

# Capgemini Most Asked Python DSA Questions

(Arrays, Strings, Stack, Queue, Trees, BST, Graphs, DP, Bit Manipulation)

Abhishek Rathor

Instagram id: SYNTAX ERROR (code.abhii07)

## Key Features:

- **75+ Interview Questions** from Capgemini coding rounds.
- **Beginner to Advanced Topics:** Arrays → Strings → Stack → Queue → Trees → BST → Graphs → DP → Bit Manipulation.
- **Code in Python** with **Time & Space Complexity**.
- **Concept Explanations** included for quick revision.
- **Optimized Solutions** for interviews and campus placements.

## Why This Guide?

- **Save Time:** No need to search multiple sources.
- **Practice Smart:** Focus on **most frequently asked questions**.
- **Learn Concepts:** Step-by-step explanations for each topic.
- **Placement Ready:** Covers all Capgemini rounds and coding challenges.

***Success is not just about writing code, it's about solving problems.***

## ARRAYS (10 Questions)

### Q1. Two Sum - Find pair with given sum

You are given an array of integers and a target sum. Find two numbers that add up to the target.

**Solution:**

```
def twoSum(nums, target):
    mp = {}
    for i in range(len(nums)):
        complement = target - nums[i]
        if complement in mp:
            return [mp[complement], i]
        mp[nums[i]] = i
    return []
```

**Explanation:** Uses hash map to store visited elements. For each element, checks if its complement exists in the map. Time complexity is  $O(n)$  with  $O(n)$  space.

---

### Q2. Maximum Subarray Sum (Kadane's Algorithm)

You are given an array of integers. Find the contiguous subarray with the maximum sum.

**Solution:**

```
def maxSubArray(nums):
    maxSum = nums[0]
    currentSum = nums[0]
    for i in range(1, len(nums)):
        currentSum = max(nums[i], currentSum + nums[i])
        maxSum = max(maxSum, currentSum)
    return maxSum
```

**Explanation:** Maintains current sum and global maximum sum. At each position, decides whether to extend current subarray or start new one. Time complexity  $O(n)$ .

---

### Q3. Rotate Array

You are given an array and an integer k. Rotate the array to the right by k steps.

**Solution:**

```
def rotate(nums, k):
    k = k % len(nums)
    nums.reverse()
    nums[:k] = reversed(nums[:k])
    nums[k:] = reversed(nums[k:])
```

**Explanation:** Uses three reversals to achieve rotation. First reverses entire array, then first k elements, then remaining elements. Time  $O(n)$ , space  $O(1)$ .

---

### Q4. Find Duplicate Number

You are given an array containing n+1 integers where each integer is between 1 and n. Find the duplicate number.

**Solution:**

```
def findDuplicate(nums):
    slow = nums[0]
    fast = nums[0]

    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break

    slow = nums[0]
    while slow != fast:
        slow = nums[slow]
        fast = nums[fast]

    return slow
```

**Explanation:** Uses Floyd's cycle detection algorithm treating array as linked list. Fast and slow pointers detect cycle, then finds cycle start which is the duplicate.  $O(n)$  time,  $O(1)$  space.

---

## Q5. Merge Sorted Arrays

You are given two sorted arrays. Merge them into a single sorted array.

**Solution:**

```
def merge(nums1, m, nums2, n):
    i = m - 1
    j = n - 1
    k = m + n - 1

    while i >= 0 and j >= 0:
        if nums1[i] > nums2[j]:
            nums1[k] = nums1[i]
            i -= 1
        else:
            nums1[k] = nums2[j]
            j -= 1
        k -= 1

    while j >= 0:
        nums1[k] = nums2[j]
        j -= 1
        k -= 1
```

**Explanation:** Merges from end to beginning to avoid overwriting elements. Compares elements from both arrays and places larger one at the end. Time  $O(m+n)$ , space  $O(1)$ .

---

## Q6. Best Time to Buy and Sell Stock

You are given an array of stock prices. Find the maximum profit from one buy and one sell transaction.

**Solution:**

```
def maxProfit(prices):
    minPrice = float('inf')
```

```
maxProfit = 0

for price in prices:
    minPrice = min(minPrice, price)
    maxProfit = max(maxProfit, price - minPrice)

return maxProfit
```

**Explanation:** Tracks minimum price seen so far and calculates profit at each point. Updates maximum profit if current profit is higher. Time  $O(n)$ , space  $O(1)$ .

---

## Q7. Move Zeros to End

You are given an array with zeros and non-zeros. Move all zeros to the end maintaining relative order.

**Solution:**

```
def moveZeroes(nums):
    j = 0
    for i in range(len(nums)):
        if nums[i] != 0:
            nums[i], nums[j] = nums[j], nums[i]
            j += 1
```

**Explanation:** Uses two pointers where j tracks position for next non-zero element. Swaps non-zero elements to front maintaining order. Time  $O(n)$ , space  $O(1)$ .

---

## Q8. Product of Array Except Self

You are given an array. Return array where each element is the product of all other elements.

**Solution:**

```
def moveZeroes(nums):
    j = 0
    for i in range(len(nums)):
        if nums[i] != 0:
```

```
nums[i], nums[j] = nums[j], nums[i]
j += 1
```

**Explanation:** First pass calculates left products, second pass calculates right products and multiplies. Avoids division and handles zeros correctly. Time  $O(n)$ , space  $O(1)$  excluding output.

---

## Q9. Find Missing Number

You are given an array containing  $n$  distinct numbers from 0 to  $n$ . Find the missing number.

**Solution:**

```
def missingNumber(nums):
    n = len(nums)
    expectedSum = n * (n + 1) // 2
    actualSum = sum(nums)
    return expectedSum - actualSum
```

**Explanation:** Calculates expected sum using formula  $n(n+1)/2$  and subtracts actual sum. The difference is the missing number. Time  $O(n)$ , space  $O(1)$ .

---

## Q10. Maximum Product Subarray

You are given an array of integers. Find the contiguous subarray with the maximum product.

**Solution:**

```
def maxProduct(nums):
    maxProd = nums[0]
    minProd = nums[0]
    result = nums[0]

    for i in range(1, len(nums)):
        if nums[i] < 0:
            maxProd, minProd = minProd, maxProd

        maxProd = max(nums[i], maxProd * nums[i])
        minProd = min(nums[i], minProd * nums[i])

    return result
```

```
        result = max(result, maxProd)

    return result
```

**Explanation:** Tracks both maximum and minimum products as negatives can become positives. Swaps when encountering negative number. Time  $O(n)$ , space  $O(1)$ .

---

## STRINGS (10 Questions)

### Q11. Valid Palindrome

You are given a string. Check if it is a palindrome considering only alphanumeric characters and ignoring case.

**Solution:**

```
def isPalindrome(s):
    left = 0
    right = len(s) - 1

    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1

        if s[left].lower() != s[right].lower():
            return False

        left += 1
        right -= 1

    return True
```

**Explanation:** Uses two pointers from both ends, skips non-alphanumeric characters. Compares characters after converting to lowercase. Time  $O(n)$ , space  $O(1)$ .

---

## Q12. Longest Substring Without Repeating Characters

You are given a string. Find the length of the longest substring without repeating characters.

**Solution:**

```
def lengthOfLongestSubstring(s):
    mp = {}
    maxLen = 0
    start = 0

    for end in range(len(s)):
        if s[end] in mp:
            start = max(start, mp[s[end]] + 1)
        mp[s[end]] = end
        maxLen = max(maxLen, end - start + 1)

    return maxLen
```

**Explanation:** Uses sliding window with hash map to track last seen positions. Moves start pointer when duplicate found. Time  $O(n)$ , space  $O(\min(m,n))$ .

---

## Q13. Valid Anagram

You are given two strings. Check if they are anagrams of each other.

**Solution:**

```
def isAnagram(s, t):
    if len(s) != len(t):
        return False

    count = {}
    for c in s:
        count[c] = count.get(c, 0) + 1

    for c in t:
        count[c] = count.get(c, 0) - 1
        if count[c] < 0:
            return False

    return True
```

**Explanation:** Counts frequency of characters in first string and decrements for second string. If any count goes negative, not an anagram. Time  $O(n)$ , space  $O(1)$ .

---

## Q14. Group Anagrams

You are given an array of strings. Group anagrams together.

**Solution:**

```
def groupAnagrams(strs):
    mp = {}

    for s in strs:
        key = ''.join(sorted(s))
        if key not in mp:
            mp[key] = []
        mp[key].append(s)

    return list(mp.values())
```

**Explanation:** Uses sorted string as key to group anagrams in hash map. All anagrams have same sorted form. Time  $O(n \cdot k \log k)$  where  $k$  is max string length.

---

## Q15. Longest Palindromic Substring

You are given a string. Find the longest palindromic substring.

**Solution:**

```
def longestPalindrome(s):
    start = 0
    maxLen = 0

    def expand(left, right):
        nonlocal start, maxLen
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1

        if right - left - 1 > maxLen:
```

```

        maxLen = right - left - 1
        start = left + 1

    for i in range(len(s)):
        expand(i, i)
        expand(i, i + 1)

    return s[start:start + maxLen]

```

**Explanation:** Expands around each center (odd and even length palindromes). Tracks longest palindrome found. Time  $O(n^2)$ , space  $O(1)$ .

---

## Q16. String to Integer (atoi)

You are given a string. Convert it to a 32-bit signed integer following atoi rules.

**Solution:**

```

def myAtoi(s):
    i = 0
    sign = 1
    result = 0
    INT_MAX = 2**31 - 1
    INT_MIN = -2**31

    while i < len(s) and s[i] == ' ':
        i += 1

    if i < len(s) and (s[i] == '+' or s[i] == '-'):
        sign = -1 if s[i] == '-' else 1
        i += 1

    while i < len(s) and s[i].isdigit():
        result = result * 10 + int(s[i])
        if result * sign > INT_MAX:
            return INT_MAX
        if result * sign < INT_MIN:
            return INT_MIN
        i += 1

```

```
return result * sign
```

**Explanation:** Handles whitespace, sign, and digits while checking overflow. Returns clamped value if overflow occurs. Time  $O(n)$ , space  $O(1)$ .

---

## Q17. Count and Say

You are given an integer  $n$ . Generate the  $n$ th term of the count-and-say sequence.

**Solution:**

```
def countAndSay(n):
    result = "1"

    for i in range(1, n):
        next_str = ""
        count = 1

        for j in range(1, len(result)):
            if result[j] == result[j-1]:
                count += 1
            else:
                next_str += str(count) + result[j-1]
                count = 1

        next_str += str(count) + result[-1]
        result = next_str

    return result
```

**Explanation:** Iteratively builds sequence by counting consecutive digits. Appends count followed by digit to next term. Time  $O(n*m)$  where  $m$  is length.

---

## Q18. Reverse Words in String

You are given a string containing words separated by spaces. Reverse the order of words.

**Solution:**

```
def reverseWords(s):
    s = list(s[::-1])
    n = len(s)
    start = 0
    end = 0

    while start < n:
        while start < n and s[start] == ' ':
            start += 1
        if start == n:
            break

        if end != 0:
            s[end] = ' '
            end += 1

        wordStart = end
        while start < n and s[start] != ' ':
            s[end] = s[start]
            start += 1
            end += 1

        s[wordStart:end] = s[wordStart:end][::-1]

    return ''.join(s[:end])
```

**Explanation:** Reverses entire string, then reverses each word individually. Removes extra spaces in-place. Time  $O(n)$ , space  $O(1)$ .

---

**Q19. Implement strStr (Find Needle in Haystack)**

You are given two strings haystack and needle. Find the first occurrence of needle in haystack.

**Solution:**

```
def strStr(haystack, needle):
    if not needle:
        return 0

    m = len(haystack)
```

```

n = len(needle)

for i in range(m - n + 1):
    j = 0
    while j < n and haystack[i + j] == needle[j]:
        j += 1

    if j == n:
        return i

return -1

```

**Explanation:** Checks each position in haystack for match with needle. Returns index of first match or -1. Time  $O(m*n)$ , can optimize with KMP.

---

## Q20. Longest Common Prefix

You are given an array of strings. Find the longest common prefix string.

**Solution:**

```

def longestCommonPrefix(strs):
    if not strs:
        return ""

    prefix = strs[0]

    for i in range(1, len(strs)):
        j = 0
        while j < len(prefix) and j < len(strs[i]) and prefix[j] ==
strs[i][j]:
            j += 1
        prefix = prefix[:j]
        if not prefix:
            break

    return prefix

```

**Explanation:** Takes first string as prefix and compares with each string. Shrinks prefix when mismatch found. Time  $O(S)$  where  $S$  is sum of all characters.

---

## STACKS (8 Questions)

### Q21. Valid Parentheses

You are given a string containing brackets. Check if brackets are properly closed and nested.

**Solution:**

```
def isValid(s):
    st = []
    pairs = {'(': ')', '[': ']', '{': '}'

    for c in s:
        if c in pairs:
            if not st or st[-1] != pairs[c]:
                return False
            st.pop()
        else:
            st.append(c)

    return len(st) == 0
```

**Explanation:** Uses stack to match opening and closing brackets. Pushes opening brackets and pops when matching closing found. Time  $O(n)$ , space  $O(n)$ .

---

### Q22. Min Stack

You are required to design a stack that supports push, pop, top, and retrieving minimum element in constant time.

**Solution:**

```
class MinStack:
    def __init__(self):
        self.st = []
        self.minSt = []

    def push(self, val):
        self.st.append(val)
        if not self.minSt or val <= self.minSt[-1]:
            self.minSt.append(val)
```

```

def pop(self):
    if self.st[-1] == self.minSt[-1]:
        self.minSt.pop()
    self.st.pop()

def top(self):
    return self.st[-1]

def getMin(self):
    return self.minSt[-1]

```

**Explanation:** Uses two stacks, one for elements and one for tracking minimums. Min stack only stores values when new minimum is found. All operations  $O(1)$ .

---

## Q23. Evaluate Reverse Polish Notation

You are given an array of strings representing arithmetic expression in Reverse Polish Notation. Evaluate it.

**Solution:**

```

def evalRPN(tokens):
    st = []

    for token in tokens:
        if token in ['+', '-', '*', '/']:
            b = st.pop()
            a = st.pop()
            if token == '+':
                st.append(a + b)
            elif token == '-':
                st.append(a - b)
            elif token == '*':
                st.append(a * b)
            else:
                st.append(int(a / b))
        else:
            st.append(int(token))

    return st[-1]

```

**Explanation:** Uses stack to store operands. When operator encountered, pops two operands and pushes result. Final stack top is answer. Time  $O(n)$ .

---

## Q24. Daily Temperatures

You are given an array of temperatures. Return array where each element is number of days until warmer temperature.

**Solution:**

```
def dailyTemperatures(temperatures):
    n = len(temperatures)
    result = [0] * n
    st = []

    for i in range(n):
        while st and temperatures[i] > temperatures[st[-1]]:
            idx = st.pop()
            result[idx] = i - idx
        st.append(i)

    return result
```

**Explanation:** Uses monotonic stack storing indices. When higher temperature found, pops and calculates days difference. Time  $O(n)$ , space  $O(n)$ .

---

## Q25. Next Greater Element

You are given an array. For each element, find the next greater element to its right.

**Solution:**

```
def nextGreaterElements(nums):
    n = len(nums)
    result = [-1] * n
    st = []

    for i in range(2 * n - 1, -1, -1):
        while st and st[-1] <= nums[i % n]:
            st.pop()
```

```

        if i < n and st:
            result[i] = st[-1]

        st.append(nums[i % n])

    return result

```

**Explanation:** Traverses array twice (circular) from right using stack. Maintains decreasing monotonic stack. Time  $O(n)$ , space  $O(n)$ .

---

## Q26. Largest Rectangle in Histogram

You are given heights of bars in histogram. Find the largest rectangle area.

**Solution:**

```

def largestRectangleArea(heights):
    st = []
    maxArea = 0
    n = len(heights)

    for i in range(n + 1):
        h = 0 if i == n else heights[i]

        while st and h < heights[st[-1]]:
            height = heights[st.pop()]
            width = i if not st else i - st[-1] - 1
            maxArea = max(maxArea, height * width)

        st.append(i)

    return maxArea

```

**Explanation:** Uses stack to track indices of increasing heights. Calculates area when smaller height encountered. Time  $O(n)$ , space  $O(n)$ .

---

## Q27. Simplify Path

You are given an absolute Unix-style file path. Simplify it to canonical path.

**Solution:**

```
def simplifyPath(path):
    st = []
    tokens = path.split('/')

    for token in tokens:
        if token == "" or token == ".":
            continue
        if token == "..":
            if st:
                st.pop()
        else:
            st.append(token)

    return "/" + "/".join(st)
```

**Explanation:** Splits path by '/' and uses stack to handle directory navigation. Handles '..' by popping and ignores '.'. Time  $O(n)$ , space  $O(n)$ .

---

## Q28. Decode String

You are given an encoded string with pattern  $k[\text{encoded\_string}]$ . Decode and return the string.

**Solution:**

```
def decodeString(s):
    countSt = []
    strSt = []
    current = ""
    k = 0

    for c in s:
        if c.isdigit():
            k = k * 10 + int(c)
        elif c == '[':
            countSt.append(k)
            strSt.append(current)
            current = ""
            k = 0
        elif c == ']':
            temp = current
            current = strSt.pop()
            temp = temp * countSt.pop()
            current += temp
```

```

        count = countSt.pop()
        current += temp * count
    else:
        current += c

return current

```

**Explanation:** Uses two stacks for counts and strings. Builds string and expands when closing bracket encountered. Time  $O(n)$ , space  $O(n)$ .

---

## QUEUES (5 Questions)

### Q29. Implement Queue using Stacks

You are required to implement a queue using only two stacks.

**Solution:**

```

class MyQueue:
    def __init__(self):
        self.s1 = []
        self.s2 = []

    def push(self, x):
        self.s1.append(x)

    def pop(self):
        if not self.s2:
            while self.s1:
                self.s2.append(self.s1.pop())
        return self.s2.pop()

    def peek(self):
        if not self.s2:
            while self.s1:
                self.s2.append(self.s1.pop())
        return self.s2[-1]

    def empty(self):
        return len(self.s1) == 0 and len(self.s2) == 0

```

**Explanation:** Uses input stack for enqueue and output stack for dequeue. Transfers elements when output stack empty. Amortized  $O(1)$  for all operations.

---

### Q30. Sliding Window Maximum

You are given an array and window size  $k$ . Find maximum in each sliding window.

**Solution:**

```
from collections import deque

def maxSlidingWindow(nums, k):
    dq = deque()
    result = []

    for i in range(len(nums)):
        while dq and dq[0] <= i - k:
            dq.popleft()

        while dq and nums[dq[-1]] < nums[i]:
            dq.pop()

        dq.append(i)

        if i >= k - 1:
            result.append(nums[dq[0]])

    return result
```

**Explanation:** Uses deque to maintain indices in decreasing order of values. Front always contains maximum of current window. Time  $O(n)$ , space  $O(k)$ .

---

### Q31. Design Circular Queue

You are required to implement a circular queue with fixed size.

**Solution:**

```
class MyCircularQueue:
    def __init__(self, k):
```

```

        self.data = [0] * k
        self.front = 0
        self.rear = -1
        self.size = 0
        self.capacity = k

    def enqueue(self, value):
        if self.isFull():
            return False
        self.rear = (self.rear + 1) % self.capacity
        self.data[self.rear] = value
        self.size += 1
        return True

    def dequeue(self):
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return True

    def Front(self):
        return -1 if self.isEmpty() else self.data[self.front]

    def Rear(self):
        return -1 if self.isEmpty() else self.data[self.rear]

    def isEmpty(self):
        return self.size == 0

    def isFull(self):
        return self.size == self.capacity

```

**Explanation:** Uses array with front and rear pointers using modulo for circular behavior. Tracks size separately for full/empty checks. All operations  $O(1)$ .

---

## Q32. First Non-Repeating Character in Stream

You are given a stream of characters. Find first non-repeating character at each step.

**Solution:**

```

from collections import deque

def firstNonRepeating(s):
    freq = {}
    q = deque()
    result = ""

    for c in s:
        freq[c] = freq.get(c, 0) + 1
        q.append(c)

        while q and freq[q[0]] > 1:
            q.popleft()

        result += '#' if not q else q[0]

    return result

```

**Explanation:** Uses queue to maintain order and map for frequency. Removes repeated characters from queue front. Time  $O(n)$ , space  $O(n)$ .

---

### Q33. Task Scheduler

You are given tasks and cooldown time  $n$ . Find minimum time to complete all tasks.

**Solution:**

```

def leastInterval(tasks, n):
    freq = {}
    maxFreq = 0

    for task in tasks:
        freq[task] = freq.get(task, 0) + 1
        maxFreq = max(maxFreq, freq[task])

    maxCount = sum(1 for f in freq.values() if f == maxFreq)

    time = (maxFreq - 1) * (n + 1) + maxCount
    return max(time, len(tasks))

```

**Explanation:** Calculates minimum intervals based on most frequent task. Formula accounts for cooldown periods between same tasks. Time  $O(n)$ , space  $O(1)$ .

---

## RECURSION (8 Questions)

### Q34. Fibonacci Number

You are given an integer  $n$ . Calculate the  $n$ th Fibonacci number using recursion.

**Solution:**

```
def fibonacci(n):
    memo = {}

    def fib(n):
        if n <= 1:
            return n
        if n in memo:
            return memo[n]
        memo[n] = fib(n-1) + fib(n-2)
        return memo[n]

    return fib(n)
```

**Explanation:** Uses memoization to store computed values avoiding recomputation. Base cases are 0 and 1. Time  $O(n)$  with memoization, space  $O(n)$ .

---

### Q35. Power Function

You are given base  $x$  and exponent  $n$ . Calculate  $x^n$  using recursion efficiently.

**Solution:**

```
def myPow(x, n):
    if n == 0:
        return 1.0

    if n < 0:
        x = 1 / x
```

```

        n = -(n + 1)
        return x * myPow(x, n)

    half = myPow(x, n // 2)
    if n % 2 == 0:
        return half * half
    return x * half * half

```

**Explanation:** Uses binary exponentiation dividing problem in half each step. Handles negative exponents by inverting base. Time  $O(\log n)$ , space  $O(\log n)$ .

---

### Q36. Generate Parentheses

You are given  $n$  pairs of parentheses. Generate all combinations of well-formed parentheses.

**Solution:**

```

def generateParenthesis(n):
    result = []

    def generate(open_count, close_count, current):
        if open_count == 0 and close_count == 0:
            result.append(current)
            return

        if open_count > 0:
            generate(open_count - 1, close_count, current + "(")

        if close_count > open_count:
            generate(open_count, close_count - 1, current + ")")

    generate(n, n, "")
    return result

```

**Explanation:** Recursively adds opening brackets when available and closing when valid. Tracks remaining open and close brackets. Time  $O(4^n/\sqrt{n})$ , space  $O(n)$ .

---

### Q37. Letter Combinations of Phone Number

You are given a string containing digits 2-9. Return all possible letter combinations from phone keypad.

#### Solution:

```
def letterCombinations(digits):
    if not digits:
        return []

    mapping = ["", "", "abc", "def", "ghi", "jkl", "mno", "pqrs",
               "tuv", "wxyz"]
    result = []

    def backtrack(idx, current):
        if idx == len(digits):
            result.append(current)
            return

        letters = mapping[int(digits[idx])]
        for c in letters:
            backtrack(idx + 1, current + c)

    backtrack(0, "")
    return result
```

**Explanation:** Uses backtracking to explore all combinations. Maps digits to letters and recursively builds strings. Time  $O(4^n)$ , space  $O(n)$ .

---

### Q38. Permutations

You are given an array of distinct integers. Return all possible permutations.

#### Solution:

```
def permute(nums):
    result = []

    def backtrack(start):
        if start == len(nums):
            result.append(nums[:])
            return

        for i in range(start, len(nums)):
            nums[start], nums[i] = nums[i], nums[start]
            backtrack(start + 1)
            nums[start], nums[i] = nums[i], nums[start]
```

```

        for i in range(start, len(nums)):
            nums[start], nums[i] = nums[i], nums[start]
            backtrack(start + 1)
            nums[start], nums[i] = nums[i], nums[start]

    backtrack(0)
    return result

```

**Explanation:** Swaps elements to generate permutations recursively. Backtracks by swapping back to original state. Time  $O(n! \cdot n)$ , space  $O(n)$ .

---

### Q39. Subsets

You are given an array of unique integers. Return all possible subsets (power set).

**Solution:**

```

def subsets(nums):
    result = []

    def backtrack(start, current):
        result.append(current[:])

        for i in range(start, len(nums)):
            current.append(nums[i])
            backtrack(i + 1, current)
            current.pop()

    backtrack(0, [])
    return result

```

**Explanation:** Generates subsets by including or excluding each element. Explores all combinations recursively. Time  $O(2^n)$ , space  $O(n)$ .

---

## Q40. Combination Sum

You are given array of candidates and target. Find all unique combinations that sum to target.

**Solution:**

```
def combinationSum(candidates, target):
    result = []

    def backtrack(start, current, target):
        if target == 0:
            result.append(current[:])
            return
        if target < 0:
            return

        for i in range(start, len(candidates)):
            current.append(candidates[i])
            backtrack(i, current, target - candidates[i])
            current.pop()

    backtrack(0, [], target)
    return result
```

**Explanation:** Uses backtracking allowing same element multiple times. Prunes when target becomes negative. Time  $O(n^{(t/m)})$ , space  $O(t/m)$ .

---

## Q41. Word Search

You are given a 2D board and a word. Check if word exists in the grid.

**Solution:**

```
def exist(board, word):
    def dfs(i, j, idx):
        if idx == len(word):
            return True

        if i < 0 or i >= len(board) or j < 0 or j >= len(board[0]) or board[i][j] != word[idx]:
            return False
```

```

        temp = board[i][j]
        board[i][j] = '#'

        found = (dfs(i+1, j, idx+1) or
                 dfs(i-1, j, idx+1) or
                 dfs(i, j+1, idx+1) or
                 dfs(i, j-1, idx+1))

        board[i][j] = temp
        return found

    for i in range(len(board)):
        for j in range(len(board[0])):
            if dfs(i, j, 0):
                return True

    return False

```

**Explanation:** Uses DFS with backtracking to explore all paths. Marks visited cells temporarily to avoid reuse. Time  $O(mn4^L)$ , space  $O(L)$ .

---

## BINARY SEARCH TREES (7 Questions)

### Q42. Validate Binary Search Tree

You are given root of binary tree. Check if it is a valid BST.

**Solution:**

```

def isValidBST(root, minVal=float('-inf'), maxVal=float('inf')):
    if not root:
        return True

    if root.val <= minVal or root.val >= maxVal:
        return False

    return (isValidBST(root.left, minVal, root.val) and
            isValidBST(root.right, root.val, maxVal))

```

**Explanation:** Recursively checks if each node value is within valid range. Left subtree must have values less than node, right greater. Time  $O(n)$ , space  $O(h)$ .

---

### Q43. Lowest Common Ancestor of BST

You are given BST and two nodes. Find their lowest common ancestor.

**Solution:**

```
def lowestCommonAncestor(root, p, q):
    if not root:
        return None

    if root.val > p.val and root.val > q.val:
        return lowestCommonAncestor(root.left, p, q)

    if root.val < p.val and root.val < q.val:
        return lowestCommonAncestor(root.right, p, q)

    return root
```

**Explanation:** Uses BST property to navigate. LCA is node where paths to p and q diverge.  
Time  $O(h)$ , space  $O(h)$ .

---

### Q44. Kth Smallest Element in BST

You are given root of BST and integer k. Find the kth smallest element.

**Solution:**

```
def kthSmallest(root, k):
    result = [-1]
    count = [k]

    def inorder(node):
        if not node:
            return

        inorder(node.left)

        count[0] -= 1
        if count[0] == 0:
            result[0] = node.val
```

```

        return

    inorder(node.right)

inorder(root)
return result[0]

```

**Explanation:** Performs inorder traversal which visits nodes in sorted order. Decrements k and returns when k becomes 0. Time  $O(h+k)$ , space  $O(h)$ .

---

## Q45. Delete Node in BST

You are given root of BST and key. Delete node with given key.

**Solution:**

```

def deleteNode(root, key):
    if not root:
        return None

    if key < root.val:
        root.left = deleteNode(root.left, key)
    elif key > root.val:
        root.right = deleteNode(root.right, key)
    else:
        if not root.left:
            return root.right
        if not root.right:
            return root.left

        minNode = root.right
        while minNode.left:
            minNode = minNode.left

        root.val = minNode.val
        root.right = deleteNode(root.right, minNode.val)

    return root

```

**Explanation:** Finds node to delete, replaces with inorder successor if has two children. Handles cases of 0, 1, or 2 children. Time  $O(h)$ , space  $O(h)$ .

---

## Q46. Convert Sorted Array to BST

You are given sorted array. Convert it to height-balanced BST.

**Solution:**

```
def sortedArrayToBST(nums):
    def helper(left, right):
        if left > right:
            return None

        mid = left + (right - left) // 2
        root = TreeNode(nums[mid])
        root.left = helper(left, mid - 1)
        root.right = helper(mid + 1, right)
        return root

    return helper(0, len(nums) - 1)
```

**Explanation:** Recursively picks middle element as root ensuring balance. Left half becomes left subtree, right half becomes right subtree. Time  $O(n)$ , space  $O(\log n)$ .

---

## Q47. Range Sum of BST

You are given BST root and range [low, high]. Find sum of values in range.

**Solution:**

```
def rangeSumBST(root, low, high):
    if not root:
        return 0

    total = 0

    if root.val >= low and root.val <= high:
        total += root.val
```

```

    if root.val > low:
        total += rangeSumBST(root.left, low, high)

    if root.val < high:
        total += rangeSumBST(root.right, low, high)

    return total

```

**Explanation:** Uses BST property to prune search space. Only explores subtrees that can contain values in range. Time  $O(n)$ , space  $O(h)$ .

---

## Q48. Insert into BST

You are given root of BST and value to insert. Insert and return modified tree.

**Solution:**

```

def insertIntoBST(root, val):
    if not root:
        return TreeNode(val)

    if val < root.val:
        root.left = insertIntoBST(root.left, val)
    else:
        root.right = insertIntoBST(root.right, val)

    return root

```

**Explanation:** Recursively finds correct position using BST property. Creates new node when nullptr reached. Time  $O(h)$ , space  $O(h)$ .

---

## TREES (9 Questions)

### Q49. Maximum Depth of Binary Tree

You are given root of binary tree. Find its maximum depth.

**Solution:**

```
def maxDepth(root):
    if not root:
        return 0

    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

**Explanation:** Recursively calculates depth of left and right subtrees. Returns maximum plus one for current node. Time  $O(n)$ , space  $O(h)$ .

---

## Q50. Symmetric Tree

You are given root of binary tree. Check if it is symmetric around its center.

**Solution:**

```
def isSymmetric(root):
    def isMirror(left, right):
        if not left and not right:
            return True
        if not left or not right:
            return False

        return (left.val == right.val and
                isMirror(left.left, right.right) and
                isMirror(left.right, right.left))

    return not root or isMirror(root.left, root.right)
```

**Explanation:** Compares left and right subtrees recursively checking mirror property. Left's left must match right's right. Time  $O(n)$ , space  $O(h)$ .

---

## Q51. Level Order Traversal

You are given root of binary tree. Return level order traversal of node values.

**Solution:**

```
from collections import deque
```

```

def levelOrder(root):
    if not root:
        return []

    result = []
    q = deque([root])

    while q:
        size = len(q)
        level = []

        for i in range(size):
            node = q.popleft()
            level.append(node.val)

            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)

        result.append(level)

    return result

```

**Explanation:** Uses BFS with queue processing nodes level by level. Tracks level size to group nodes. Time  $O(n)$ , space  $O(n)$ .

---

## Q52. Diameter of Binary Tree

You are given root of binary tree. Find length of longest path between any two nodes.

**Solution:**

```

def diameterOfBinaryTree(root):
    diameter = [0]

    def height(node):
        if not node:
            return 0

        left = height(node.left)
        right = height(node.right)

```

```
        diameter[0] = max(diameter[0], left + right)

    return 1 + max(left, right)

height(root)
return diameter[0]
```

**Explanation:** Calculates height while tracking maximum path length through each node. Diameter is sum of left and right heights. Time  $O(n)$ , space  $O(h)$ .

---

### Q53. Invert Binary Tree

You are given root of binary tree. Invert it (mirror reflection).

**Solution:**

```
def invertTree(root):
    if not root:
        return None

    temp = root.left
    root.left = invertTree(root.right)
    root.right = invertTree(temp)

    return root
```

**Explanation:** Recursively swaps left and right children at each node. Base case returns nullptr for leaf children. Time  $O(n)$ , space  $O(h)$ .

---

### Q54. Path Sum

You are given root and target sum. Check if there's root-to-leaf path summing to target.

**Solution:**

```
def hasPathSum(root, targetSum):
    if not root:
        return False

    if not root.left and not root.right:
```

```
        return root.val == targetSum

    return (hasPathSum(root.left, targetSum - root.val) or
            hasPathSum(root.right, targetSum - root.val))
```

**Explanation:** Recursively subtracts node values from target. Returns true when leaf reached with remaining sum equal to leaf value. Time  $O(n)$ , space  $O(h)$ .

---

## Q55. Lowest Common Ancestor of Binary Tree

You are given binary tree and two nodes. Find their lowest common ancestor.

**Solution:**

```
def lowestCommonAncestor(root, p, q):
    if not root or root == p or root == q:
        return root

    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)

    if left and right:
        return root

    return left if left else right
```

**Explanation:** Recursively searches for nodes in left and right subtrees. If found in both, current node is LCA. Time  $O(n)$ , space  $O(h)$ .

---

## Q56. Binary Tree Right Side View

You are given root of binary tree. Return values visible from right side.

**Solution:**

```
from collections import deque

def rightSideView(root):
    if not root:
```

```

        return []

    result = []
    q = deque([root])

    while q:
        size = len(q)

        for i in range(size):
            node = q.popleft()

            if i == size - 1:
                result.append(node.val)

            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)

    return result

```

**Explanation:** Performs level order traversal and picks last node of each level. Uses BFS with queue. Time  $O(n)$ , space  $O(n)$ .

---

## Q57. Serialize and Deserialize Binary Tree

You are required to design algorithm to serialize and deserialize binary tree.

**Solution:**

```

class Codec:
    def serialize(self, root):
        if not root:
            return "null,"

        return str(root.val) + "," + self.serialize(root.left) + self.serialize(root.right)

    def deserialize(self, data):
        def helper(data_list):
            val = data_list.pop(0)

```

```

        if val == "null":
            return None

        root = TreeNode(int(val))
        root.left = helper(data_list)
        root.right = helper(data_list)
        return root

    data_list = data.split(',')
    return helper(data_list)

```

**Explanation:** Uses preorder traversal for serialization with null markers. Deserializes by parsing string recursively. Time  $O(n)$ , space  $O(n)$ .

---

## GRAPHS (8 Questions)

### Q58. Number of Islands

You are given 2D grid of '1's (land) and '0's (water). Count number of islands.

**Solution:**

```

def numIslands(grid):
    def dfs(i, j):
        if i < 0 or i >= len(grid) or j < 0 or j >= len(grid[0]) or \
            grid[i][j] == '0':
            return

        grid[i][j] = '0'
        dfs(i+1, j)
        dfs(i-1, j)
        dfs(i, j+1)
        dfs(i, j-1)

    count = 0
    for i in range(len(grid)):
        for j in range(len(grid[0])):
            if grid[i][j] == '1':
                count += 1
                dfs(i, j)

```

```
return count
```

**Explanation:** Uses DFS to mark all connected land cells. Each DFS call represents one island discovery. Time  $O(mn)$ , space  $O(mn)$ .

---

## Q59. Clone Graph

You are given reference of node in connected undirected graph. Return deep copy.

**Solution:**

```
def cloneGraph(node):
    if not node:
        return None

    visited = {}

    def clone(node):
        if node in visited:
            return visited[node]

        copy = Node(node.val)
        visited[node] = copy

        for neighbor in node.neighbors:
            copy.neighbors.append(clone(neighbor))

        return copy

    return clone(node)
```

**Explanation:** Uses DFS with hash map to track cloned nodes. Creates copy and recursively clones neighbors. Time  $O(n+e)$ , space  $O(n)$ .

---

## Q60. Course Schedule (Cycle Detection)

You are given number of courses and prerequisites. Check if you can finish all courses.

**Solution:**

```

from collections import deque

def canFinish(numCourses, prerequisites):
    graph = [[] for _ in range(numCourses)]
    indegree = [0] * numCourses

    for pre in prerequisites:
        graph[pre[1]].append(pre[0])
        indegree[pre[0]] += 1

    q = deque()
    for i in range(numCourses):
        if indegree[i] == 0:
            q.append(i)

    count = 0
    while q:
        course = q.popleft()
        count += 1

        for next_course in graph[course]:
            indegree[next_course] -= 1
            if indegree[next_course] == 0:
                q.append(next_course)

    return count == numCourses

```

**Explanation:** Uses Kahn's algorithm (topological sort) to detect cycles. If all courses processed, no cycle exists. Time  $O(v+e)$ , space  $O(v+e)$ .

---

## Q61. Pacific Atlantic Water Flow

You are given  $m \times n$  matrix representing heights. Find cells where water can flow to both oceans.

**Solution:**

```

def pacificAtlantic(heights):
    if not heights:
        return []

    m, n = len(heights), len(heights[0])

```

```

pacific = [[False] * n for _ in range(m)]
atlantic = [[False] * n for _ in range(m)]

def dfs(i, j, visited, prev):
    if (i < 0 or i >= m or j < 0 or j >= n or
        visited[i][j] or heights[i][j] < prev):
        return

    visited[i][j] = True
    dfs(i+1, j, visited, heights[i][j])
    dfs(i-1, j, visited, heights[i][j])
    dfs(i, j+1, visited, heights[i][j])
    dfs(i, j-1, visited, heights[i][j])

for i in range(m):
    dfs(i, 0, pacific, -1)
    dfs(i, n-1, atlantic, -1)

for j in range(n):
    dfs(0, j, pacific, -1)
    dfs(m-1, j, atlantic, -1)

result = []
for i in range(m):
    for j in range(n):
        if pacific[i][j] and atlantic[i][j]:
            result.append([i, j])

return result

```

**Explanation:** DFS from ocean edges inward checking reachability. Cells reachable from both oceans are added to result. Time  $O(mn)$ , space  $O(mn)$ .

---

## Q62. Shortest Path in Binary Matrix

You are given  $n \times n$  binary matrix. Find shortest clear path from top-left to bottom-right.

**Solution:**

```

from collections import deque

def shortestPathBinaryMatrix(grid):

```

```

if grid[0][0] == 1:
    return -1

n = len(grid)
dirs = [[-1,-1],[-1,0],[-1,1],[0,-1],[0,1],[1,-1],[1,0],[1,1]]
q = deque([(0, 0)])
grid[0][0] = 1
steps = 1

while q:
    size = len(q)
    for _ in range(size):
        x, y = q.popleft()

        if x == n-1 and y == n-1:
            return steps

        for dx, dy in dirs:
            nx, ny = x + dx, y + dy
            if 0 <= nx < n and 0 <= ny < n and grid[nx][ny] == 0:
                grid[nx][ny] = 1
                q.append((nx, ny))

    steps += 1

return -1

```

**Explanation:** Uses BFS to find shortest path considering 8 directions. Marks visited cells to avoid revisiting. Time  $O(n^2)$ , space  $O(n^2)$ .

---

## Q63. Rotting Oranges

You are given grid with fresh and rotten oranges. Find minimum time for all oranges to rot.

**Solution:**

```

from collections import deque

def orangesRotting(grid):
    m, n = len(grid), len(grid[0])
    q = deque()
    fresh = 0

```

```

for i in range(m):
    for j in range(n):
        if grid[i][j] == 2:
            q.append((i, j))
        if grid[i][j] == 1:
            fresh += 1

if fresh == 0:
    return 0

minutes = 0
dirs = [[-1,0],[1,0],[0,-1],[0,1]]

while q:
    size = len(q)
    for _ in range(size):
        x, y = q.popleft()

        for dx, dy in dirs:
            nx, ny = x + dx, y + dy
            if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] == 1:
                grid[nx][ny] = 2
                fresh -= 1
                q.append((nx, ny))

    minutes += 1

return minutes - 1 if fresh == 0 else -1

```

**Explanation:** Uses multi-source BFS starting from all rotten oranges. Spreads rot level by level tracking time. Time  $O(mn)$ , space  $O(mn)$ .

---

## Q64. Word Ladder

You are given beginWord, endWord and wordList. Find shortest transformation sequence length.

**Solution:**

```

from collections import deque

```

```

def ladderLength(beginWord, endWord, wordList):
    word_set = set(wordList)
    if endWord not in word_set:
        return 0

    q = deque([beginWord])
    level = 1

    while q:
        size = len(q)
        for _ in range(size):
            word = q.popleft()

            if word == endWord:
                return level

            for i in range(len(word)):
                original = word[i]
                for c in 'abcdefghijklmnopqrstuvwxyz':
                    word_list = list(word)
                    word_list[i] = c
                    new_word = ''.join(word_list)

                    if new_word in word_set:
                        q.append(new_word)
                        word_set.remove(new_word)

        level += 1

    return 0

```

**Explanation:** Uses BFS trying all one-letter transformations at each level. Erases used words to avoid cycles. Time  $O(m^2n)$ , space  $O(mn)$ .

---

## Q65. Graph Valid Tree

You are given  $n$  nodes and edges list. Check if edges form valid tree.

**Solution:**

```

def validTree(n, edges):
    if len(edges) != n - 1:

```

```

        return False

parent = list(range(n))

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

for edge in edges:
    p1 = find(edge[0])
    p2 = find(edge[1])

    if p1 == p2:
        return False

    parent[p1] = p2

return True

```

**Explanation:** Uses union-find to detect cycles. Valid tree has  $n-1$  edges and no cycles. Time  $O(n \cdot \alpha(n))$ , space  $O(n)$ .

---

## DYNAMIC PROGRAMMING (5 Questions)

### Q66. Climbing Stairs

You are given  $n$  steps. Find number of ways to climb using 1 or 2 steps at a time.

**Solution:**

```

def climbStairs(n):
    if n <= 2:
        return n

    prev2 = 1
    prev1 = 2

    for i in range(3, n + 1):
        current = prev1 + prev2
        prev2 = prev1
        prev1 = current

```

```
    return prev1
```

**Explanation:** Uses DP similar to Fibonacci. Ways to reach step  $i$  is sum of ways to reach  $i-1$  and  $i-2$ . Time  $O(n)$ , space  $O(1)$ .

---

## Q67. House Robber

You are given array of house values. Find maximum money you can rob without robbing adjacent houses.

**Solution:**

```
def rob(nums):
    if not nums:
        return 0

    prev2 = 0
    prev1 = 0

    for num in nums:
        current = max(prev1, prev2 + num)
        prev2 = prev1
        prev1 = current

    return prev1

}
```

**Explanation:** At each house, chooses max between robbing current house plus  $i-2$  or skipping it. Uses two variables for optimization. Time  $O(n)$ , space  $O(1)$ .

---

## Q68. Coin Change

You are given coins array and amount. Find minimum coins needed to make amount.

**Solution:**

```
def coinChange(coins, amount):
    dp = [amount + 1] * (amount + 1)
    dp[0] = 0
```

```

for i in range(1, amount + 1):
    for coin in coins:
        if i >= coin:
            dp[i] = min(dp[i], dp[i - coin] + 1)

return dp[amount] if dp[amount] <= amount else -1

```

**Explanation:** Builds up solutions from 0 to amount. For each amount, tries all coins and picks minimum. Time  $O(\text{amount} \times \text{coins})$ , space  $O(\text{amount})$ .

---

## Q69. Longest Increasing Subsequence

You are given array of integers. Find length of longest strictly increasing subsequence.

**Solution:**

```

from bisect import bisect_left

def lengthOfLIS(nums):
    dp = []

    for num in nums:
        pos = bisect_left(dp, num)
        if pos == len(dp):
            dp.append(num)
        else:
            dp[pos] = num

    return len(dp)

```

**Explanation:** Uses binary search with patience sorting approach. Maintains array of smallest tail elements for each length. Time  $O(n \log n)$ , space  $O(n)$ .

---

## Q70. Unique Paths

You are given  $m \times n$  grid. Find number of unique paths from top-left to bottom-right.

**Solution:**

```

from bisect import bisect_left

def lengthOfLIS(nums):
    dp = []

    for num in nums:
        pos = bisect_left(dp, num)
        if pos == len(dp):
            dp.append(num)
        else:
            dp[pos] = num

    return len(dp)

```

**Explanation:** Uses 1D DP array where each cell is sum of paths from left and top. Space optimized from 2D to 1D. Time  $O(m*n)$ , space  $O(n)$ .

## BIT MANIPULATION (5 Questions)

### Q71. Single Number

You are given array where every element appears twice except one. Find the single element.

**Solution:**

```

def singleNumber(nums):
    result = 0
    for num in nums:
        result ^= num
    return result

```

**Explanation:** XOR of two same numbers is 0, and XOR of any number with 0 is number itself. All pairs cancel out. Time  $O(n)$ , space  $O(1)$ .

### Q72. Number of 1 Bits (Hamming Weight)

You are given unsigned integer. Count number of '1' bits.

**Solution:**

```
def hammingWeight(n):  
    count = 0  
    while n:  
        count += 1  
        n &= (n - 1)  
    return count
```

**Explanation:** Brian Kernighan's algorithm flips rightmost set bit to 0 each iteration. Counts iterations until n becomes 0. Time  $O(\log n)$ , space  $O(1)$ .

---

### Q73. Reverse Bits

You are given 32-bit unsigned integer. Reverse its bits.

**Solution:**

```
def reverseBits(n):  
    result = 0  
    for i in range(32):  
        result <<= 1  
        result |= (n & 1)  
        n >>= 1  
    return result
```

**Explanation:** Extracts least significant bit of n and adds to result. Shifts result left and n right 32 times. Time  $O(1)$ , space  $O(1)$ .

---

### Q74. Power of Two

You are given integer n. Check if it is a power of two.

**Solution:**

```
def isPowerOfTwo(n):  
    return n > 0 and (n & (n - 1)) == 0
```

**Explanation:** Power of 2 has only one bit set.  $n \& (n-1)$  removes rightmost bit, result is 0 for powers of 2. Time  $O(1)$ , space  $O(1)$ .

---

## Q75. Missing Number (Bit Manipulation)

You are given array containing  $n$  distinct numbers from 0 to  $n$ . Find missing number using XOR.

**Solution:**

```
def missingNumber(nums):
    result = len(nums)
    for i in range(len(nums)):
        result ^= i ^ nums[i]
    return result
```

**Explanation:** XORs all indices and values. Duplicate numbers cancel out leaving only missing number.

Time  $O(n)$ , space  $O(1)$ .

## **Congratulations!**

You have successfully completed this DSA & Tech guide. This marks a step forward in your coding journey! Remember, learning never stops—keep practicing, solving problems, and building projects. Every line of code you write brings you closer to becoming a master of Data Structures, Algorithms, and modern tech skills.

Stay connected with us for more **coding tutorials, challenges, tips, and tech content** on Instagram: **@SYNTAXERROR**. Let's grow together as a community of passionate coders!

**© 2025-26 SYNTAX ERROR. All Rights Reserved.**

This content is exclusively created by SYNTAX ERROR. Unauthorized copying, sharing, or redistribution of this DSA & Tech content is strictly prohibited. Protecting intellectual property ensures quality learning for everyone and maintains the integrity of this work.